Network Programming

Lecture slides - I

UNIT 1

Network Programming Fundamentals

This lecture will cover:

1.1 Introduction and importance of networking and network programming.
1.2 Fundamentals of Client server and P2P models
1.3 Basic Network Protocols
    1.3.1 Characteristics and use cases of TCP, IP, UDP, SCTP
1.4 TCP state transition diagram.
1.5 Comparisons of protocols (TCP,UDP,SCTP)
1.6 Introduction to Sockets
    1.6.1 Concept of sockets in network communication
    1.6.2 Types of sockets: Stream (TCP) vs Datagram (UDP)

Introduction

Computer Networking and Network Programming .

1.Computer Networking:

Networking primarily refers to the practice of designing, implementing, managing, and maintaining computer networks. It focuses on the physical and logical connections that enable communication between different devices and systems. Networking involves setting up the infrastructure that allows devices to communicate with each other, share resources, and access the internet. It encompasses a wide range of activities, including:

- **Network Design**: Planning and designing the layout of the network, including the hardware components (routers, switches, cables) and the logical structure (IP addressing, subnets, VLANs).
- **Network Configuration**: Setting up and configuring network devices such as routers, switches, firewalls, and access points to ensure proper communication and security.
- **Network Management**: Monitoring network performance, troubleshooting issues, optimizing network resources, and ensuring the network operates efficiently.
- **Network Security**: Implementing security measures such as firewalls, encryption, access control, and intrusion detection to protect the network from unauthorized access and cyber threats.

Introduction

Computer Networking and Network Programming .

2.Network Programming

Network programming, on the other hand, refers to the practice of writing software programs that communicate over a computer network. It involves developing applications that can send and receive data over the network, allowing devices to exchange information and services. Network programming typically involves using networking protocols and APIs to enable communication between devices. Key aspects of network programming include:

•**Socket Programming**: Creating network sockets that enable communication between applications running on different devices. Sockets allow programs to establish connections, send data, and receive data over a network.

•**Protocol Implementation**: Implementing network protocols such as TCP/IP, UDP, HTTP, FTP, etc., to facilitate communication between applications.

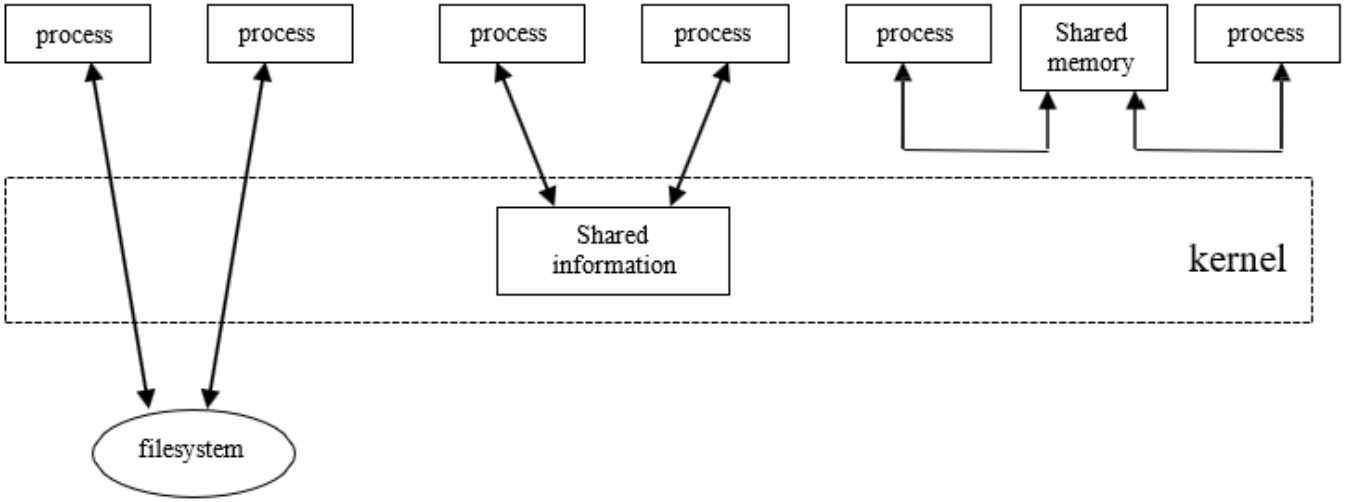
•**Client-Server Architecture**: Developing client-server applications where one program (client) requests services or resources from another program (server) over the network.

•**Data Serialization**: Encoding data into a format that can be transmitted over the network and decoded by the receiving end.

In summary, network programming focuses on developing software applications that utilize networking capabilities to enable communication and data exchange between devices

Introduction

Computer network programming involves writing computer programs that enable process to communicate with each other across a computer network or within same system in order to share information and resources.



Three ways to share information between UNIX processes

Introduction

- The two processes share some information that resides in a **file in the filesystem**. To access this data, each process must go through the kernel (e.g. read, write, lseek, etc). In order to protect multiple writers from each other, and to protect one or more readers from a writer, some form of synchronization is required when a file is being updated.

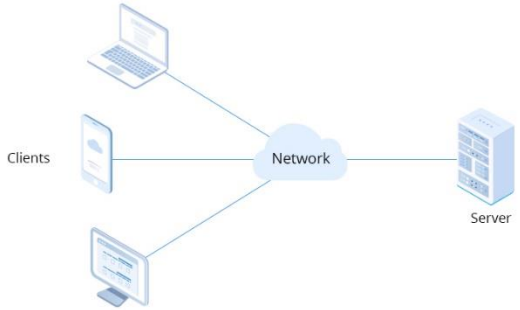
- The two processes share some information **that resides within the kernel**. Pipe, System V message queues and System V Semaphores are the examples of this type of sharing. Each operation has to access the shared information now involves a system call into the kernel.

- The two processes have a region of **shared memory** that each process can reference. Once the shared memory is set up by each process, the processes can access the data in the shared memory without involving the kernel at all. Some form of synchronization is required by the processes that are sharing the memory.

Introduction

Client Server Model

- In a **client–server network**, there is at least one dedicated central server that controls the network, and a number of clients connect to the server to carry out specific tasks.
- A client-server network **can have more than one central server**, each performing a specific function. Functions may include user access, data storage, internet connection management, and network traffic monitoring, among others.
- **Multiple clients connect to one central server.** A client is a computer or computer-controlled device that provides users with access to data on the remote server. Types of clients include smartphones, desktop computers, and laptops.
- This network architecture facilitates efficient communication and resource sharing between the central server and connected clients, enabling seamless task execution and data access across various devices



Introduction

Client Server Model

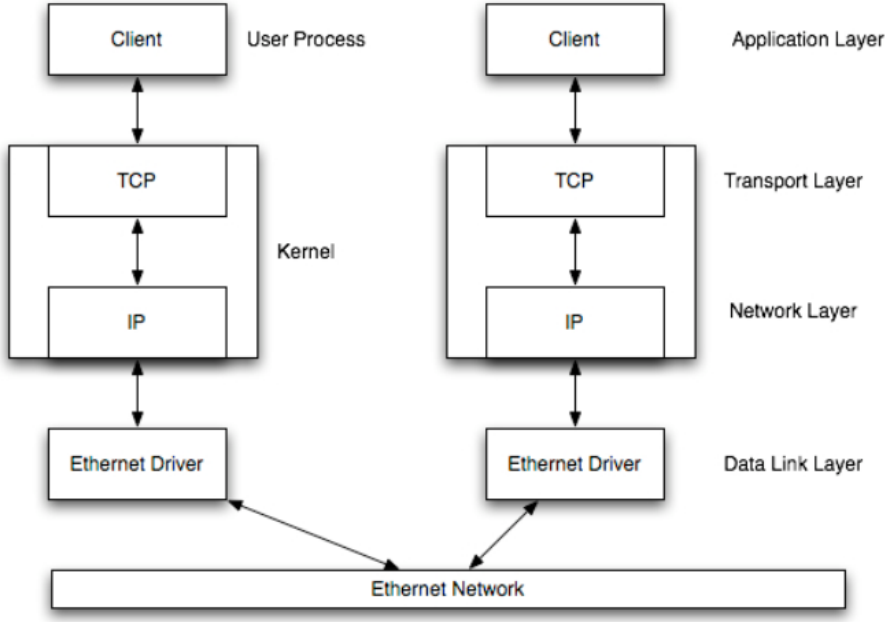


Figure 1: Client and server on the same Ethernet communicating using TCP/IP.

Introduction

Client Server Model

Benefits of a Client-Server Network

Centralized Management:

Streamlined server management allows IT teams to update data files centrally, enhancing accessibility for users. Monitoring data from a single server enables the anticipation of potential problems.

Scalability:

With minimal downtime, a client-server network can be grown by adding servers, PCs, and network segments. Scalability is a feature of client-server networks. The user has the ability to increase the quantity of resources, such as servers and clients, as needed. As a result, expanding the server's capacity can be done without causing any problems.

Enhanced Security:

Storing critical information on a single server, rather than across multiple clients, provides better protection against external threats, ensuring a heightened level of security.

Easy Management:

Proximity is not a concern for clients and the server in client-server networks. Streamlined file handling is achieved with centralized storage, offering superior tools for easy tracking and locating essential files.

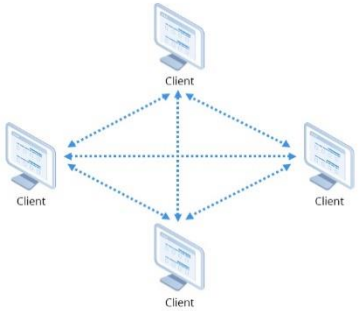
Seamless Feature Integration:

New features can be seamlessly added to a server without disrupting the normal operations of other devices within the network.

Introduction

P2P Model

- In a peer-to-peer network, there is **no central server controlling the network**. Instead, all the computers in the network are connected to one another and share resources such as files, applications, and programs. In a peer-to-peer network, each computer can function as either a client or a server, enabling it to request or provide services
- Each computer is referred to as a peer and possesses the same capabilities and access rights. **No peer has control over another.** For example, a printer connected to one computer can be utilized by any other computer on the network.
- Because resource management and network security are not controlled centrally, it is essential to have local backups for each computer. Peers can communicate directly with each other, and there are no restrictions when adding a device to a peer-to-peer network."



Introduction

P2P Model

Benefits of a Peer-to-Peer Network

Streamlined file sharing:

Files can be easily shared over long distances and accessed at any time in an advanced peer-to-peer network.

Flexibility and scalability:

New clients can be effortlessly added, enhancing the network's flexibility and scalability.

Optimized Resource Usage:

Peer-to-peer networks excel in resource utilization by distributing the workload among multiple peers. Each peer contributes its resources, including bandwidth, storage, and processing power, which are then shared across the network. This collaborative resource sharing optimizes utilization and lessens the burden on individual participants.

Cost-effective setup:

No investment in central servers is required, and there's no need for a full-time system administrator.

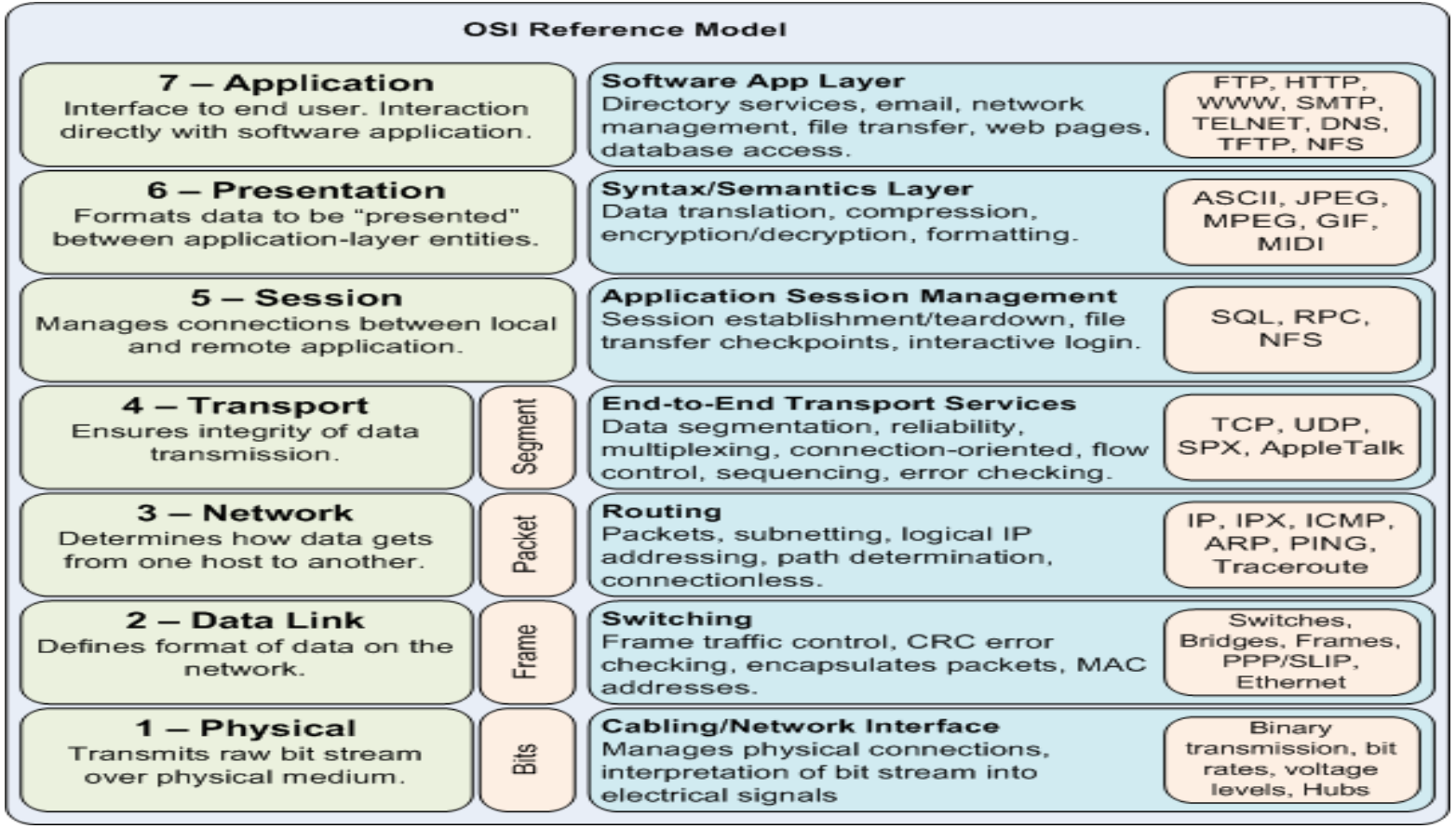
Fault tolerance:

In the event of a single computer malfunction, other computers in the peer-to-peer network continue to operate, preventing traffic bottlenecks.

Improved Privacy and Security:

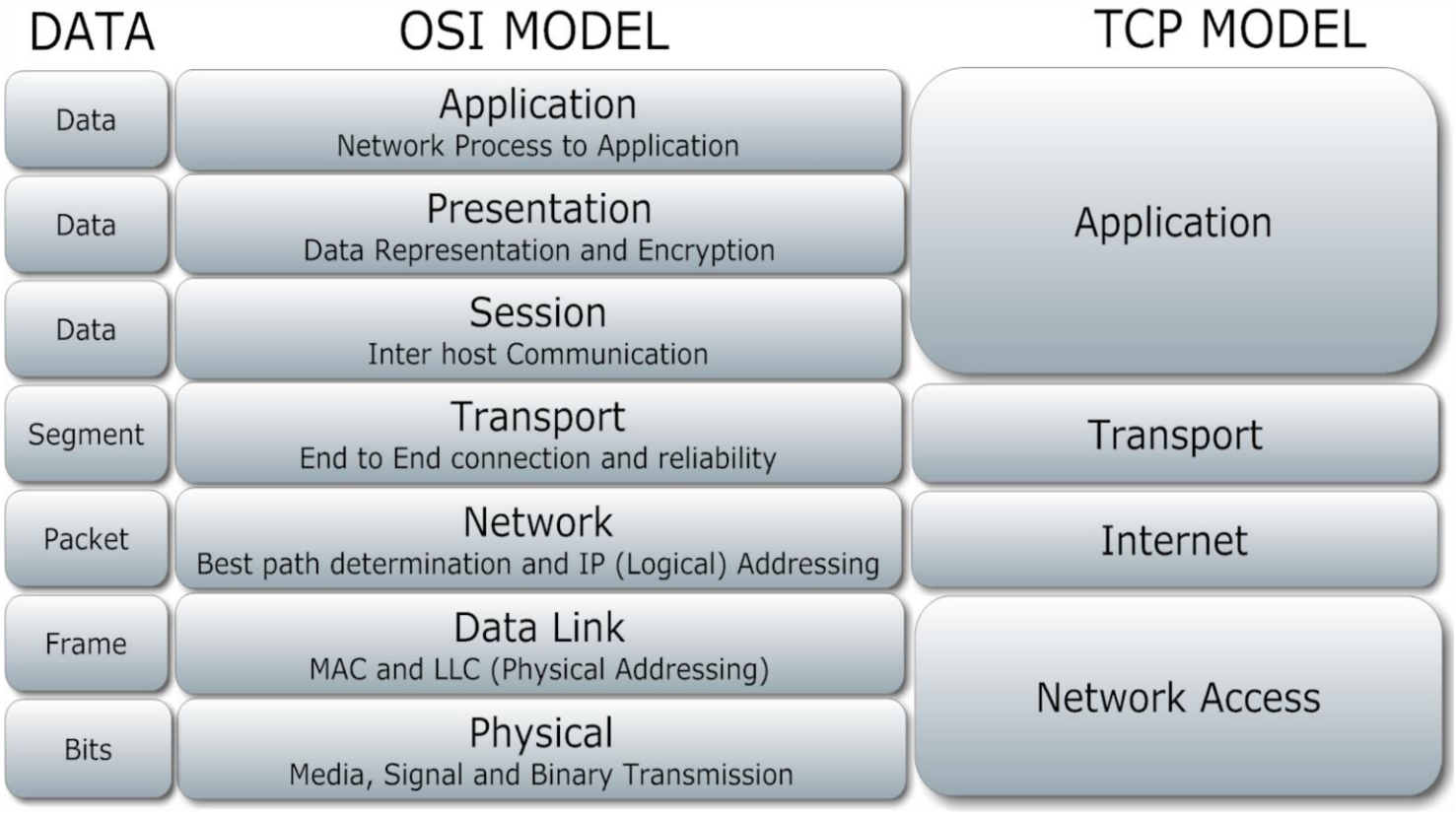
Peer-to-Peer networks contribute to enhanced privacy and security. Peer-to-peer communication can be encrypted, guaranteeing the confidentiality of data exchanged within the network. The decentralized structure, without a central server, minimizes vulnerability to single-point attacks, enhancing resilience against malicious activities.

Introduction



Introduction

OSI VS TCP/IP



Introduction

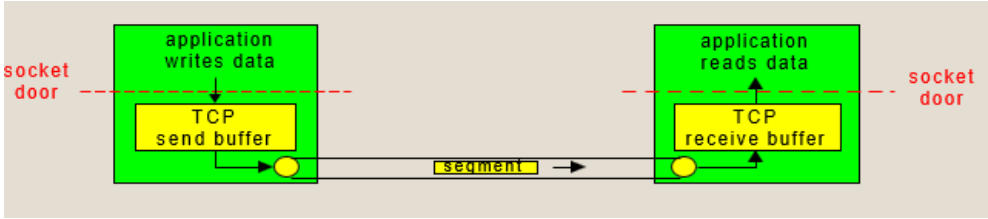
OSI and TCP/IP Similarities

1. Both the reference models are based upon **layered architecture.**
2. The physical layer and the data link layer of the OSI model correspond to the link layer of the TCP/IP model. The network layers and the transport layers are the same in both the models. The session layer, the presentation layer and the application layer of the OSI model together form the application layer of the TCP/IP model.
3. In both the models, **protocols** are defined in a layer-wise manner.
4. In both models, data is divided into packets and each packet may take the individual route from the source to the destination.

Introduction

TCP: Overview

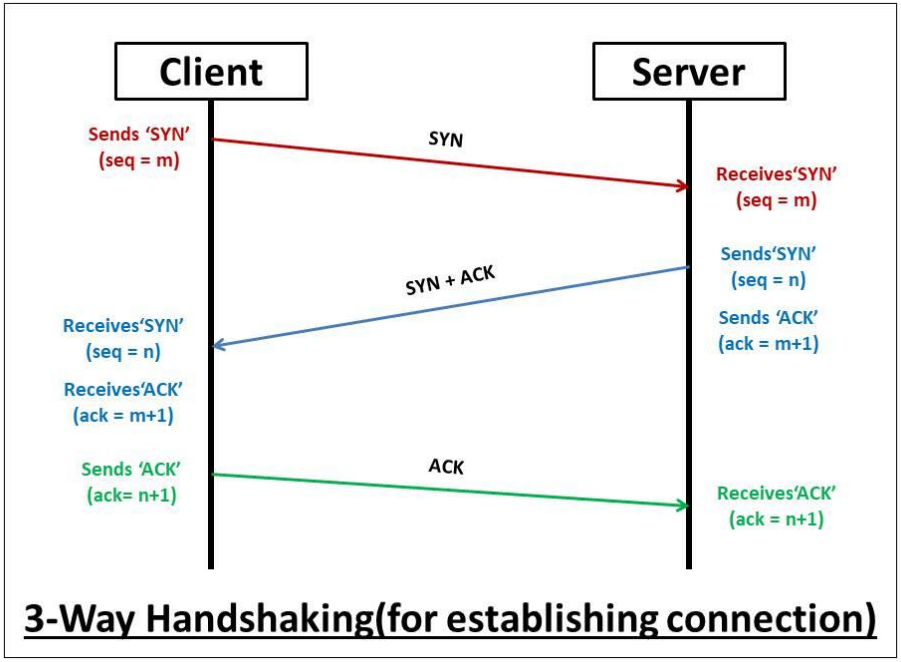
- point-to-point (unicast):
  - one sender, one receiver
- connection-oriented:
  - handshaking (exchange of control msgs) init's sender, receiver state before data exchange
  - State resides only at the END systems – Not a virtual circuit!
- full duplex data:
  - bi-directional data flow in same connection (A->B & B->A in the same connection)
  - MSS: maximum segment size



- reliable, in-order *byte steam:*
  - no "message boundaries"
- send & receive buffers
  - buffer incoming & outgoing data
- flow controlled:
  - sender will not overwhelm receiver
- congestion controlled:
  - sender will not overwhelm network

Introduction

TCP Transaction



A TCP Transaction consists of 3 Phases

1. Connection Establishment

Handshaking between client and server

2. Reliable, In-Order Data Exchange

Recover any lost data through retransmissions and ACKs

3. Connection Termination

Closing the connection

Introduction

User Datagram Protocol (UDP)

- UDP is a simple **transport-layer protocol**. (RFC 768)
- The application writes a message to a **UDP socket**, which is then encapsulated in a UDP **datagram**, which is then further encapsulated as an IP datagram, which is then sent to its destination.
- There is **no guarantee** that a **UDP datagram** will ever reach its final destination, that order will be preserved across the network, or that datagrams arrive only once.
- With network programming using UDP is its **lack of reliability**. If a datagram reaches its final destination but the checksum detects an error, or if the datagram is dropped in the network, it is not delivered to the UDP socket and is **not automatically retransmitted.**
- Each UDP datagram has a length. The length of a datagram is passed to the receiving application along with the data.
- UDP provides a **connectionless** service, as there need not be any long-term relationship between a UDP client and server.

Introduction

Stream Control Transmission Protocol (SCTP)

1. Like TCP, SCTP provides **reliability, sequencing, flow control**, and full- **duplex data transfer.**
2. Unlike TCP, SCTP provides:
   a. **Association** instead of "connection": An association refers to a communication between two systems, which may involve more than two addresses due to **multihoming.**
   b. **Message-oriented:** provides sequenced delivery of individual records. Like UDP, the length of a record written by the sender is passed to the receiving application.
   c. **Multihoming:** allows a single SCTP endpoint to support multiple IP addresses. This feature can provide increased robustness against network failure.(If one path fails, SCTP can automatically switch to another — improving reliability.)
   Both the client and the server exchange their complete lists of available local IP addresses. The single port number, however, is shared across all these addresses for that association.

Introduction

Transmission Control Protocol (TCP)

1. **Connection**: TCP provides connections between clients and servers. A TCP client establishes a connection with a server, exchanges data across the connection, and then terminates the connection.
2. **Reliability**: TCP requires acknowledgment when sending data. If an acknowledgment is not received, TCP automatically retransmits the data and waits a longer amount of time.
3. **Round-trip time** (RTT): TCP estimates RTT between a client and server dynamically so that it knows how long to wait for an acknowledgment.
4. **Sequencing**: TCP associates a sequence number with every byte (**segment**, unit of data that TCP passes to IP.) it sends. TCP reorders out-of-order segments and discards duplicate segments.
5. **Flow control:** is a set of procedures to restrict the amount of data that sender can send. Stop and Wait Protocol is a **flow control** protocol where sender sends one data packet to the receiver and then stops and waits for its acknowledgement from the receiver.
6. **Full-duplex**: an application can send and receive data in both directions on a given connection at any time.

Introduction

TCP Connection Establishment

The following scenario occurs when a TCP connection is established.

1. The server must be prepared to accept an incoming connection. This is normally done by calling **socket, bind**, and **listen** and is called a passive open.
2. The client issues an active open by calling **connect**. This causes the client TCP to send a "**synchronize**" (**SYN**) segment, which tells the server the client's initial sequence number for the data that the client will send on the connection. Normally, there is no data sent with the **SYN**; it just contains an IP header, a TCP header, and possible TCP options.
3. The server must acknowledge (**ACK**) the client's **SYN** and the server must also send its own **SYN** containing the initial sequence number for the data that the server will send on the connection. The server sends its **SYN** and the **ACK** of the client's **SYN** is a single segment.
4. The client must acknowledge the server's SYN.

Introduction

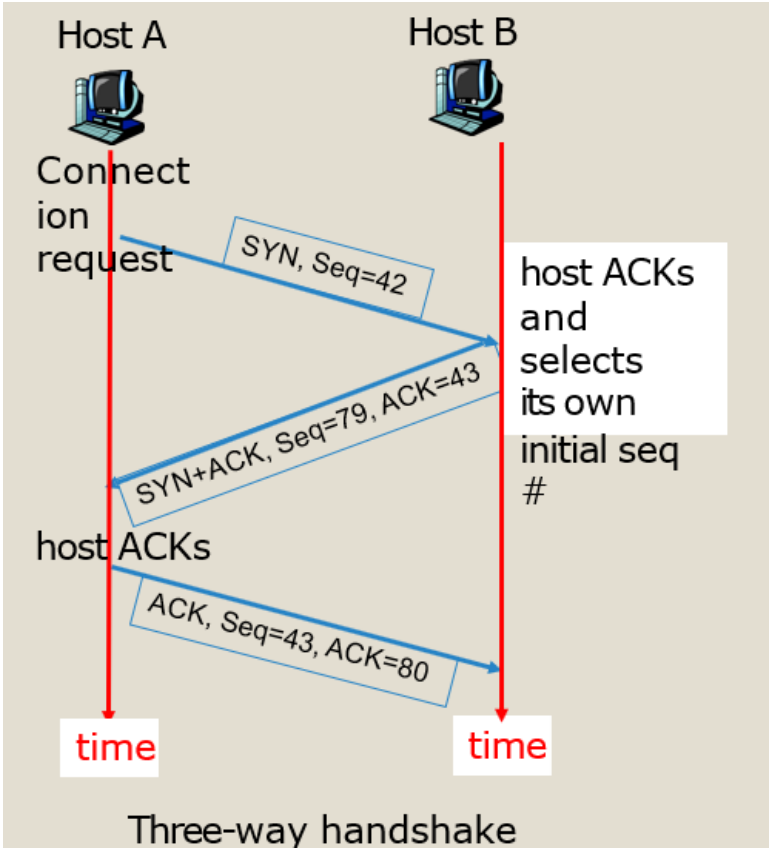
TCP Connection Establishment

Three way handshake:

Step 1: client host sends TCP SYN segment to server
- specifies a random initial seq #
- no data

Step 2: server host receives SYN, replies with SYNACK segment
- server allocates buffers
- specifies server initial seq. #

Step 3: client receives SYNACK, replies with ACK segment, which may contain data



Introduction

TCP Connection Termination

1. One application calls **close** first, and we say that this end performs the **active close**. This end's TCP sends a **FIN** segment, which means it is **finished sending data**.
2. The other end that receives the **FIN** performs the **passive close**. The received FIN is acknowledged by **TCP**. The receipt of the **FIN** is also passed to the application as an end-of-file, since the receipt of the **FIN** means the application will not receive any additional data on the connection.
3. Sometime later, the application that received the end-of-file will close its socket. This causes its **TCP** to send a **FIN**.
4. The TCP on the system that receives this final **FIN** (the end that did the active close) acknowledges the **FIN**.

Introduction

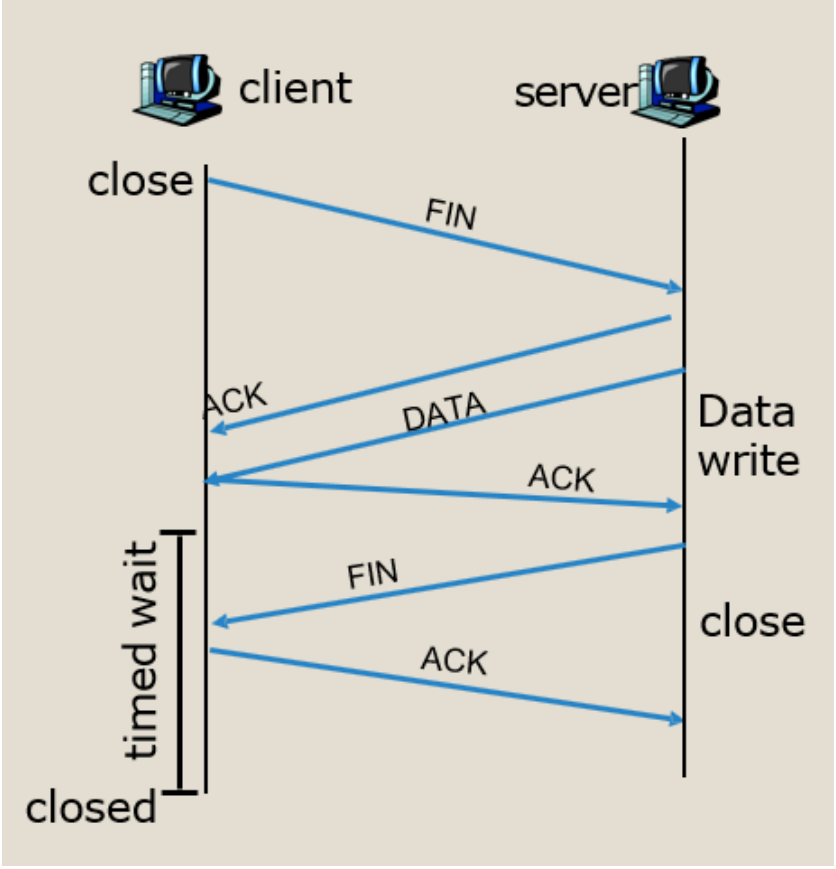
TCP Connection Termination

client closes socket:

```
clientSocket.close();
```

Step 1: client end system sends TCP FIN control segment to server

Step 2: server receives FIN, replies with ACK. Server might send some buffered but not sent data before closing the connection. Server then sends FIN and moves to Closing state.



Introduction

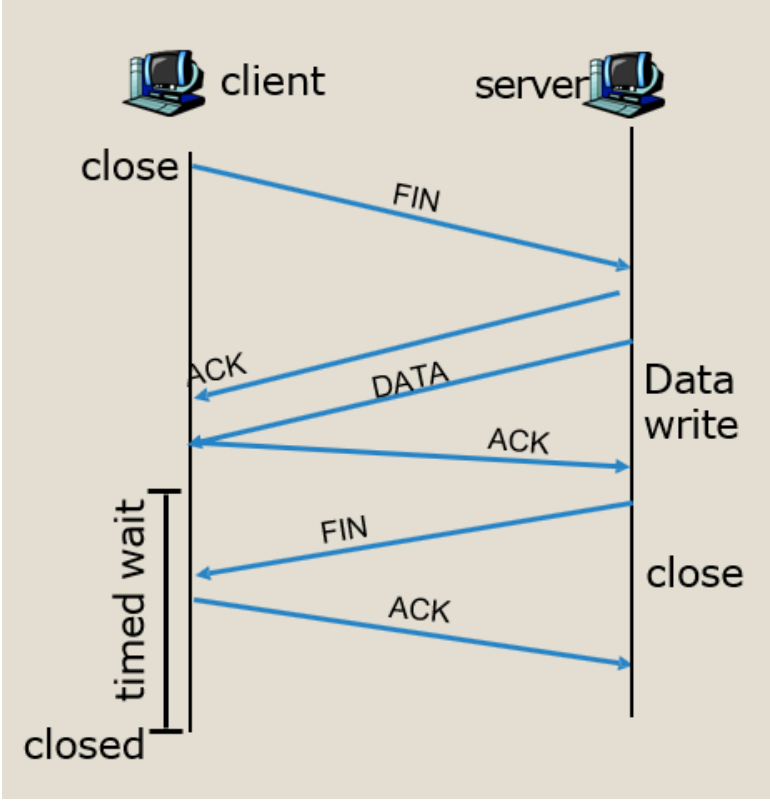
TCP Connection Termination

Step 3: client receives FIN, replies with ACK.

- Enters "timed wait" - will respond with ACK to received FINs

Step 4: server, receives ACK. Connection closed.

- Why wait before closing the connection?
  - If the connection were allowed to move to **CLOSED** state, then another pair of application processes might come along and open the same connection (use the same port #s) and a delayed FIN from an earlier would terminate the connection.



Introduction

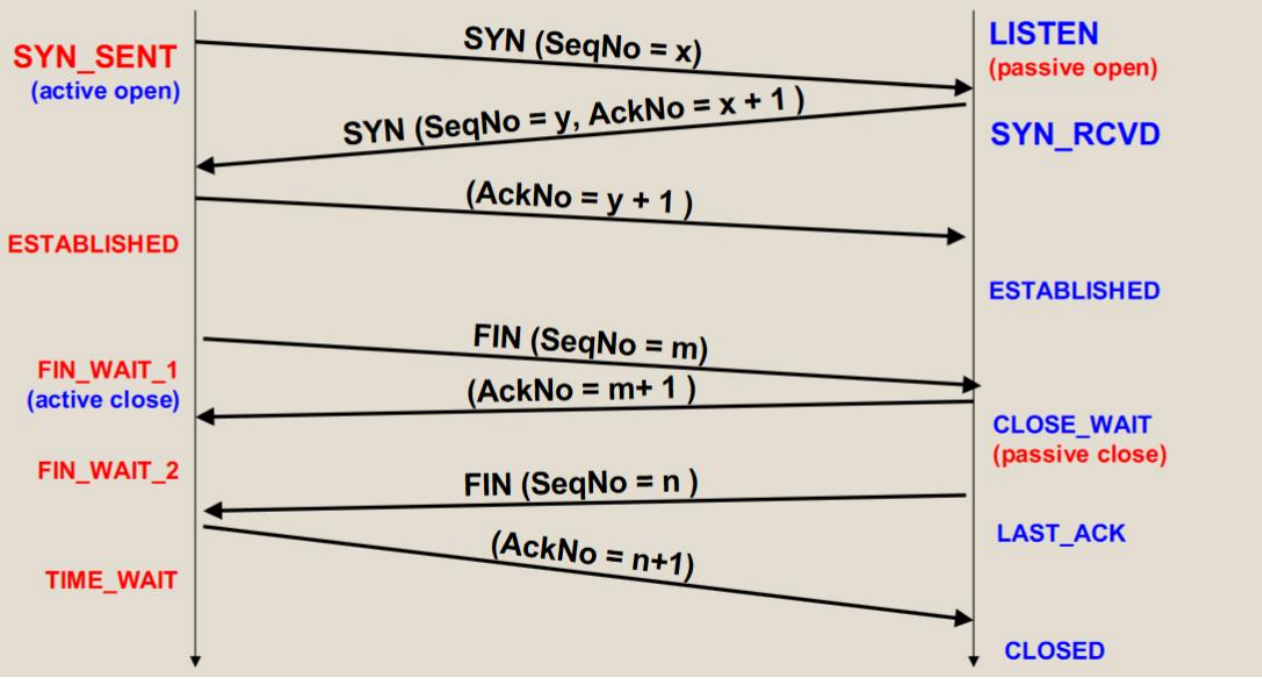
TCP/IP STATE TRANSITION DIAGRAM

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for Ack
SYN SENT	The client has started to open a connection
ESTABLISHED	Normal data transfer state
FIN WAIT 1	Client has said it is finished
FIN WAIT 2	Server has agreed to release
TIMED WAIT	Wait for pending packets (“2MSL wait state”)
CLOSING	Both Sides have tried to close simultanesously
CLOSE WAIT	Server has initiated a release
LAST ACK	Wait for pending packets

Introduction

TCP/IP STATE TRANSITION DIAGRAM

TCP States in "Normal" Connection Lifetime



Active open refers to a client initiating a connection to a server by sending a request, while

Passive open involves a server waiting for incoming connection requests

Introduction

TCP Connection Establishment and Termination

Three-Way Handshake:

When a **TCP connection is established**, the following scenario occurs:

1. The server must be prepared to accept an incoming connection. This is normally done by called socket, bind & listen. It is **known as a passive open.**
2. The client issues an active open by calling connect. This causes the client TCP to send a SYN segment (synchronize) to tell the server about the client's initial sequence number for the data. Normally, there is no data sent with the SYN; it just contains an IP header, a TCP header, and possible TCP options.
3. The server must acknowledge the client's SYN and the server must also send its own SYN containing the initial sequence number. The sever sends its SYN and the ACK of the client's SYN in a sing segment.
4. The client must acknowledge the server's SYN.

Introduction

TCP Connection Establishment and Termination

TCP Connection Termination:

While it takes three segments to establish a connection, it takes four segments to terminate connection.

1. A peer calls “close”, which refers that this end performs the “active close”. This end’s TCP sends a FIN segment, which means it **is finished sending data**.
2. The other peer that receives Fin performs the “**passive close**”. The receipt of the FIN is also passed to the application as an end-of-file, which means the application will never receive any additional data on the connection, except those that may already be queued for the application to receive.
3. The application that received the end-of-file will “**close**” its socket. This causes its TCP to send a FIN.
4. The TCP on the system that receives this final FIN acknowledges the FIN.

Introduction

Protocol Comparison TCP/IP vs UDP vs SCTP

Protocol	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)	SCTP (Stream Control Transmission Protocol)
Reliability	Reliable data delivery with error detection, retransmission, and acknowledgement mechanisms	Unreliable data delivery without error recovery or acknowledgement	Reliable data delivery with error detection, retransmission, and acknowledgement mechanisms
Connection Type	Connection-oriented	Connectionless	Connection-oriented
Ordering	Guarantees ordered delivery of data packets	Does not guarantee the ordered delivery of data packets	Guarantees ordered delivery of data packets
Speed	Slower due to reliability mechanisms	Faster due to minimal overhead	Comparable to TCP, slower than UDP due to additional functionality

Introduction

Protocol Comparison TCP/IP vs UDP vs SCTP

Applications	Web browsing, email transfer, file transfer (FTP)	Real-time communication, video streaming, online gaming, DNS	Telecommunications, voice and video over IP, signalling transport
Congestion Control	Implements congestion control mechanisms to optimize network performance	No congestion control mechanisms	Implements congestion control mechanisms to optimize network performance
Error Recovery	Detects and retransmits lost or corrupted packets	No error recovery mechanisms	Detects and retransmits lost or corrupted packets
Message-Oriented Delivery	No	No	Yes, supports message-oriented delivery

Sockets

How should two hosts communicate with each other over the Internet?

- The “Internet Protocol” (IP)
- Transport protocols: TCP, UDP

How should programmers interact with the protocols?

Sockets API – application programming interface

Sockets API: provides set of system calls that allow applications to communicate over a network using protocols like TCP and UDP

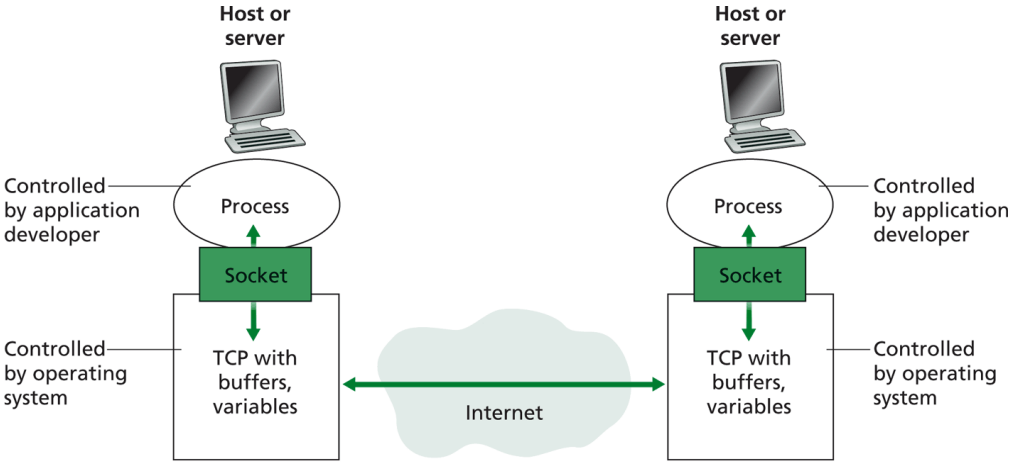
- ❑ An interface to the transport layer
- ❑ Introduced in 1981 by BSD 4.1(Berkeley Software Distribution)
- ❑ Implemented as **library and/or system calls**
- ❑ Similar interfaces to TCP and UDP
- ❑ Can also serve as interface to IP (for super- user); known as “raw sockets”

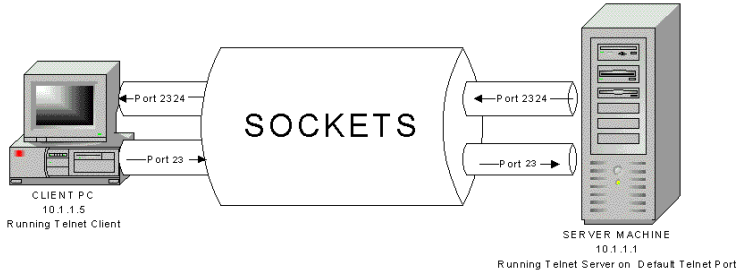
Sockets

Socket is an abstraction for an end point of communication that can be manipulated with a file descriptor.

- **File descriptor:** File descriptors are **nonnegative** integers that the kernel uses to identify the file being accessed by a particular process. Whenever the kernel opens an existing file or creates a new file it returns the file descriptor. (provides description on devices, sockets, etc. also)
- A socket is an **abstraction** which provides an **endpoint** for communication between processes.
- A **socket address** is the combination of an IP address (the location of the computer) and a port (a specific service) into a single identity.
- **Interprocess communication** consists of transmitting a message between a socket in **one process** and a socket in **another process.**
- A communication domain is an abstraction introduced to bundle common properties of processes communicating through sockets. Example: UNIX domain, internet domain. Communication domain groups sockets by their addressing and communication method.

Sockets





Messages sent to a particular Internet address and port number can be receive only by a process whose socket is associated with that Internet address and port number.

- Processes may use the **same socket** for **sending** and **receiving** messages.
- Any process may make use of **multiple ports to receive messages**, BUT a process cannot share ports with other processes on the same computer.
- **Each socket** is associated with a **particular protocol**, either UDP or TCP.

Sockets

Socket is an **abstraction** that is provided to an application programmer to send or receive data to another process.

- Data can be sent to or received from another process running on the same machine or a different machine.

What is a socket?

- To the **kernel**, a socket is an **endpoint of communication**.
- To an **application**, a socket is **a file descriptor** that lets the application **read/write from/to the network.**

Remember: All Unix I/O devices, including networks, are modeled as files.

- Clients and servers communicate with each by reading from and writing to socket descriptors.

Sockets API

- A socket API is an application-programming interface (API), usually provided by the **operating system**, that allows application programs to control and use network sockets.
- Internet socket APIs are usually based on the **Berkeley sockets standard**.
- In the Berkeley sockets standard, sockets are a form of file descriptor (a file handle)
- **In inter-process communication**, each end will generally have **its own socket,** but these may use different APIs: they are abstracted by the network protocol.
- A **socket address** is the **combination of an IP address and a port number.**

Sockets API

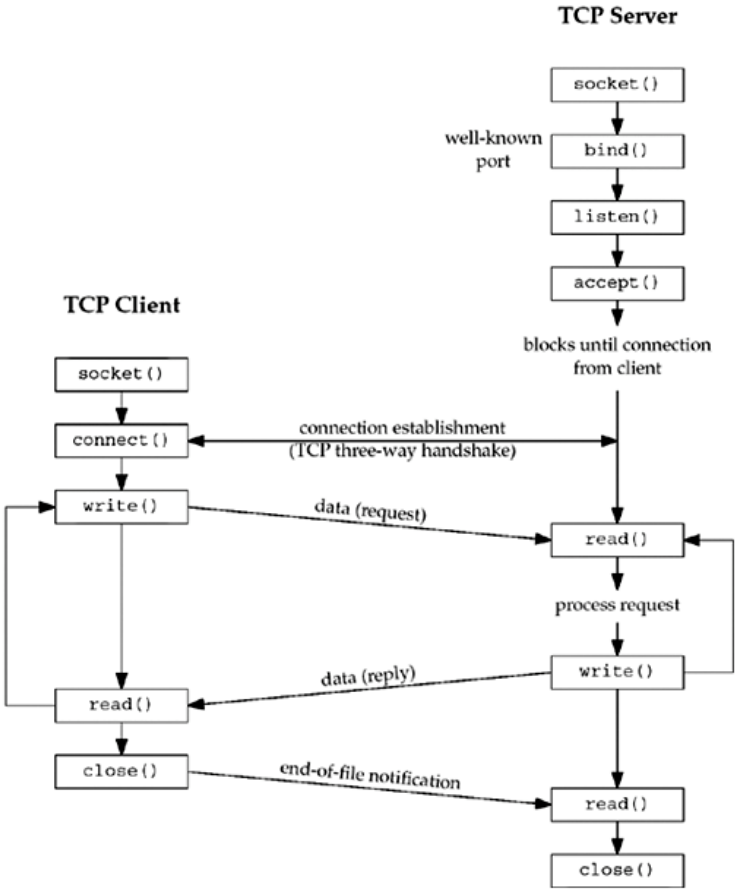
Berkeley Sockets

- The Berkeley sockets **application programming interface (API)** comprises a library for developing applications in the C programming language that perform inter-process communication, most commonly across a computer network.

- Berkeley sockets (also known as the **BSD socket API**) originated with the 4.2BSD Unix operating system (released in 1983) as an API. Only in 1989, however, could UC Berkeley release versions of its operating system and networking library free from the licensing constraints of AT&T's copyright-protected Unix.

- Many Unix systems started with some version of the BSD networking code, including the **sockets API**, and we refer to these implementations as Berkeley-derived implementations.

Socket functions for TCP client/server



Socket Address Structure

- Various structures are used in Unix Socket Programming to **hold information about the address and port,** and other information.

- Most **socket functions** require a **pointer to a socket address structure** as an argument.
- The names of these structures begin with “sockaddr_” with a distinct suffix for each protocol suite. A “sockaddr_in” structure contains an “in_addr” structure as a member field.
- **Generic socket** address structure to **hold socket information**. It is passed in most of the socket **function calls**.

- Unix Socket Programming provides **IP version specific socket** address structures

Socket Address Structure

IPv4 Socket Address Structure

- An IPv4 socket address structure is also known as "**Internet socket address structure**", which is named as "sockaddr_in" and is defined by including the "<netinet/in.h>" header. The structure is as follows:

```
struct in_addr {
    in_addr_t s_addr;              /* 32-bit IPv4 address */
                                   /* network byte ordered */
};

struct sockaddr_in {
    uint8_t sin_len;               /* length of structure (16) */
    sa_family_t sin_family;        /* AF_INET */
    in_port_t sin_port;            /* 16-bit TCP/UDP port number */
                                   /* network byte ordered */

struct in_addr sin_addr;           /* 32-bit IPv4 address */
                                   /*network byte ordered*/
char sin_zero[8];                  /* unused */ };
```

Address Family :
AF_INET (IP Version 4)

Type of socket

1. **Datagram Sockets:** Datagram sockets allow processes to use the User Datagram Protocol (UDP). It is a two-way flow of communication or messages. It can receive messages in a different order from the sending way and also can receive duplicate messages. These sockets are preserved with their boundaries. The socket type of datagram socket is SOCK_DGRAM.

- Provides datagrams, which are **connectionless messages** of a **fixed maximum length**. This type of socket is generally used for short messages, such as a name server or time server, because the order and reliability of message delivery is not guaranteed.

- In the UNIX domain, the **SOCK_DGRAM** socket type is similar to a message queue. In the Internet domain, the **SOCK_DGRAM** socket type is implemented on the User Datagram Protocol/Internet Protocol (UDP/IP) protocol.

- A ***datagram* socket** supports the bidirectional flow of data, which is not sequenced, reliable, or unduplicated. A process receiving messages on a datagram socket may find messages duplicated or in an order different than the order sent. Record boundaries in data, however, are preserved. Datagram sockets closely model the facilities found in many contemporary packet-switched networks.

Type of socket

2. Stream Sockets: Stream socket allows processes to use the Transfer Control Protocol (TCP) for communication. A stream socket provides a sequenced, constant or reliable, and two-way (bidirectional) flow of data. After the establishment of connection, data can be read and written to these sockets in a byte stream. The socket type of stream socket is SOCK_STREAM

- Provides sequenced, **two-way byte streams** with a transmission mechanism for stream data. This socket type transmits data on a reliable basis, in order, and with out-of-band capabilities. In the UNIX domain, the **SOCK_STREAM** socket type works like a pipe. In the Internet domain, the **SOCK_STREAM** socket type is implemented on the Transmission Control Protocol/Internet Protocol (TCP/IP) protocol.

- A ***stream* socket** provides for the bidirectional, reliable, sequenced, and unduplicated flow of data without record boundaries. Aside from the bidirectionality of data flow, a pair of connected stream sockets provides an interface nearly identical to pipes.

End of Chapter